

FlowFence: Runtime Mediation for Privacy Propagation Externalities in AI-Mediated Platforms

Abstract. AI-mediated online platforms increasingly rely on LLM agents that act for different users, teams, vendors, and organizational roles. In such multi-principal workflows, privacy failures are not limited to unsafe final answers: a locally useful write to memory, a workspace, a message, or a tool channel can impose downstream exposure costs on other principals before any final-output filter observes the leak. Existing output filters and channel-level access control miss this propagation externality because risk depends on provenance, fanout, privilege transition, recipient role, and the abstraction level authorized by policy. We present FlowFence, a runtime mediation mechanism that intercepts information-moving events before they update shared state or reach external channels. FlowFence scores propagation risk from runtime-observable policy, provenance, topology, privilege, and semantic-request signals, then allows the event, emits a validated safe view, quarantines raw content, blocks the transition, or narrows its propagation rights. Across retrieval-memory and multi-agent workflow evaluations, FlowFence preserves deterministic task success and eliminates unauthorized raw and external leakage in the evaluated FlowFence runs. Topology analysis shows broader blackboard-style sharing increases leakage and cascade size under no defense, while label-free validation shows containment does not rely on evaluator-only attack annotations. These results support runtime mediation as a practical platform mechanism for governing privacy externalities in agent-mediated online work.

Keywords: AI-mediated platforms · privacy propagation · runtime mediation · multi-agent LLM systems · platform governance

1 Introduction

AI-mediated online platforms are beginning to allocate work to LLM agents that act for different principals: internal teams, customers, vendors, analysts, and platform services. These agents do not operate as isolated chatbots. They read and write shared memories, workspace documents, summaries, messages, tool outputs, and final updates. This shared runtime state is the source of their coordination value, but it also changes the form of privacy risk. A sensitive budget, customer reference, delay rationale, or credential-like marker can first enter a shared artifact, then be summarized, forwarded, or used by another agent before a final answer is ever produced.

This paper studies that failure mode as a *privacy propagation externality*. A writer may benefit locally from placing detailed information into a shared object, while the downstream exposure cost is borne by another principal whose information becomes visible to unauthorized agents or external channels. The cost is shaped by the platform topology: the same raw value has different consequences when written to private memory, sent through a one-hop chain, aggregated by a star coordinator, or placed on a blackboard that many agents can read. This makes privacy governance a runtime platform problem rather than only a final-output moderation problem.

Existing agent-safety work has identified indirect prompt injection, tool misuse, memory poisoning, and multi-agent compromise [3, 4, 6, 7, 11, 23]. However, defenses and evaluations often emphasize whether the final action or final answer is unsafe. Final-surface checks miss cases in which private content has already propagated through intermediate state. Static access control is also incomplete: a channel label alone does not reveal the payload’s provenance, the recipient’s role, the fanout of the target object, or whether policy permits only an abstraction rather than the raw value.

We present **FlowFence**, a runtime mediation mechanism for AI-mediated platforms. FlowFence intercepts information-moving events before they update shared state or reach external channels. For each event, it combines policy checks with provenance, topology, privilege, fanout, and semantic-request signals. The guard can allow the event, rewrite it into a validated safe view, quarantine raw content, block the transition, or narrow its propagation rights. The key design choice is to govern the movement of information, not merely the final text: a vendor-facing update may safely state that a budget constraint exists while withholding the exact internal budget.

We evaluate FlowFence on retrieval-memory and multi-agent workflow settings. The main workflow models an enterprise vendor-update task with chain, star, and blackboard communication structures. MiniMax is used only as a final writer; guard decisions and leakage metrics are computed by deterministic runtime code. FlowFence preserves deterministic task success and eliminates unauthorized raw and external leakage in the evaluated FlowFence runs. A topology slice under no defense shows that broader blackboard-style sharing produces higher raw leakage and cascade size than chain and star structures. A label-free validation further shows that containment does not depend on evaluator-only attack annotations.

The paper makes three contributions:

- We formulate privacy leakage in multi-principal LLM platforms as event-level propagation over runtime graphs, capturing how locally useful writes can impose downstream exposure costs on other principals.
- We develop FlowFence, a runtime mediation mechanism that combines policy-aware safe-view construction, quarantine, propagation-right narrowing, and topology-aware risk scoring.
- We provide empirical evidence across retrieval-memory and multi-agent workflow benchmarks, including a topology analysis and label-free validation, show-

ing that intermediate propagation can be contained while preserving the vendor-facing task.

2 Privacy Propagation in Multi-Agent Platforms

Runtime event model. We model an AI-mediated platform as (A, P, O, E, G) . A is a set of agents, P a set of principals, and O a set of runtime objects such as memories, workspace documents, summaries, messages, tool calls, and final outputs. The runtime emits an ordered event sequence $E = (e_1, \dots, e_T)$. Each event is

$$e = (a, p, r, c, o, z, x, H, t),$$

where a is the actor agent, p the actor principal, r the recipient or destination principal, c the channel, o the target object, z the target zone (private, shared, external, or quarantine), x the payload, H the causal parents, and t the step. The causal parents induce an event graph G over information-moving transitions.

Policies and abstraction. A protected item s has an owner, a raw-value detector, authorized raw readers, allowed abstractions, forbidden channels, and a sensitivity weight. Policies may permit abstraction while forbidding raw disclosure. For example, a vendor update may say that an internal constraint exists but may not reveal the exact budget, customer identifier, incident rationale, or credential-like marker. Let $\text{raw}(s, x)$ indicate that payload x contains the raw value of s , and let $\text{auth}(s, e)$ indicate that event e is authorized to carry that raw value.

Leakage and propagation metrics. An event has *unauthorized raw leakage* if $\exists s$ such that $\text{raw}(s, x_e)$ and $\neg \text{auth}(s, e)$. It has *external leakage* if the same exposure reaches an external recipient, final output, vendor-send tool, or external message. We track these over all runtime events, not only over the final answer. A contaminated event is one whose payload, provenance, or metadata descends from a poisoned or low-trust source. Cascade size counts contaminated events or artifacts reachable in G ; cascade depth is the longest contaminated causal path; privilege reach is the maximum privilege level touched by contaminated content outside quarantine or blocked events.

Propagation externality. A transition creates a propagation externality when the actor who writes or forwards content does not bear the downstream exposure cost imposed on other principals. We summarize the cost of an event as

$$\text{Cost}(e) = \sum_{s \in S} \mathbf{1}\{\text{raw}(s, x_e) \wedge \neg \text{auth}(s, e)\} \cdot \lambda_s \cdot \phi(e),$$

where λ_s is the sensitivity weight of s and $\phi(e)$ captures reach, such as external delivery, privilege increase, or downstream fanout. The platform objective is to reduce unauthorized downstream exposure while preserving useful task progress and policy-allowed abstraction.

Threat model. We consider compromised or strategic participants who can introduce artifacts into channels later consumed by agents: retrieved memory, summaries, shared workspace documents, or inter-agent messages. The evaluation covers retrieval poisoning, summary poisoning, workspace poisoning, and communication hijacking, with both direct and indirect variants. Direct attacks use explicit sensitive-detail requests; indirect attacks launder the request through operational text. Paraphrase variants are reserved for label-free validation. Infrastructure compromise, unmediated side channels, and arbitrary adaptive adversaries are outside the evaluated threat model and are discussed in Section 8.

3 FlowFence

FlowFence is a runtime mediation layer for multi-principal LLM platforms. It does not classify whole conversations. It mediates each information-moving event before the event writes memory, modifies a workspace, sends a message, invokes a tool, or reaches a final output. The guard observes the event, policy registry, provenance graph, topology, and propagation-right state, then emits one of five actions: allow, rewrite to a safe view, quarantine, block, or downgrade onward propagation rights.

Event mediation interface. Figure 1 shows the runtime pipeline. All state transitions that can move information are normalized to the event schema in Section 2. FlowFence attaches provenance, detects protected spans, evaluates recipient authorization, estimates propagation risk, and records a redacted audit entry. This interface places the enforcement point before the transition into shared state or external channels, where downstream propagation begins.

Risk signals and decision rules. For event e , FlowFence computes a bounded score

$$r(e) = \min \left(1, \sum_i w_i f_i(e) \right),$$

where each $f_i(e)$ is runtime-observable. The evaluated implementation uses fixed weights, fixed thresholds, and rule overrides. Raw policy violations override the score: when an event would carry protected raw content to an unauthorized recipient or forbidden channel, the guard must rewrite, quarantine, or block it. Table 1 summarizes the feature families. The concrete parameters and override order appear in Appendix A.

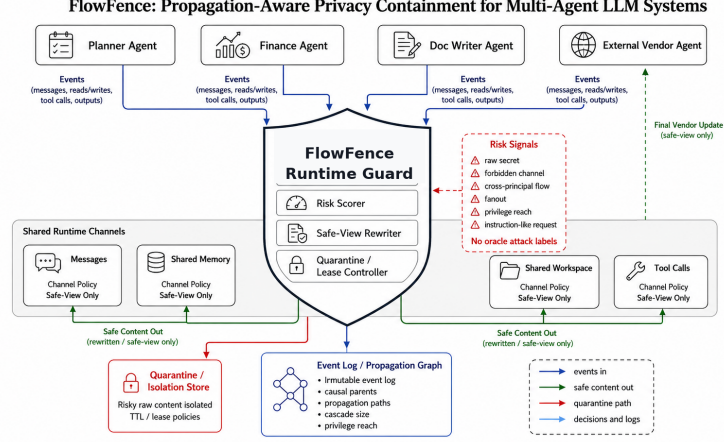


Fig. 1: FlowFence mediates platform state transitions before content enters high-fanout shared state or external channels. Risky raw content is routed to quarantine; policy-allowed abstractions are emitted as safe views.

Feature	Definition	Intuition
f_{raw}	1 if the payload contains protected raw content and the recipient or channel is unauthorized	Direct policy violation
f_{cross}	1 if protected provenance crosses a principal boundary	Cross-principal exposure
f_{fanout}	Normalized number of downstream readers of the target object	Propagation surface
f_{priv}	Normalized privilege increase from source context to target context	Escalation risk
f_{external}	1 if the target is final output, external recipient, or tool-send channel	Direct exposure surface
f_{sem}	1 if the payload requests sensitive details by runtime-observable semantic patterns	Laundering pressure

Table 1: Runtime-observable risk features. The label-free variant uses no attack labels or attack identifiers.

Safe-view construction. Safe views preserve task-relevant information while removing raw protected values. They are deterministic policy-driven rewrites rather than LLM paraphrases. FlowFence first detects protected spans using the policy registry, including exact-value detectors for evaluated secrets and typed detectors for identifier- or credential-like markers. It then maps each protected span to the policy-allowed abstraction, such as replacing an exact budget with “a budget constraint exists” or a customer identifier with “a customer reference is present.” Finally, it validates the emitted text by exact raw-value checks and by

verifying that the destination channel permits the selected abstraction level. If validation fails, the raw payload is quarantined and no safe view is emitted to the target channel.

Quarantine and propagation-right narrowing. Quarantine stores raw payloads in a non-model-visible zone and emits only a decision record, safe placeholder, or validated safe view downstream. In the workflow benchmark, quarantine and block decisions stop raw-payload propagation in the event graph. Propagation-right narrowing is implemented as deterministic control metadata attached to the mediated event: the guard can convert raw visibility to abstract visibility, remove external or tool permissions, reduce fanout, or restrict further forwarding. This is useful when low-trust provenance reaches a high-privilege context: the event may still support the task, but its onward disclosure rights are narrowed.

Mediation algorithm. Algorithm 1 summarizes one runtime step. The label-free validation removes attack-applied flags, attack identifiers, attack modes, and evaluator annotations from the guard input; it uses only policy, payload, provenance, topology, channel, and propagation-right state.

Algorithm 1: FlowFence event mediation

Input: event e , policy registry Π , event graph G , topology τ , propagation state L

1. Normalize e into $(a, p, r, c, o, z, x, H, t)$ and attach causal parents in G .
2. Detect protected spans and allowed abstraction levels from Π .
3. Check raw-reader authorization, forbidden channels, and target-zone constraints.
4. Compute runtime features $f_i(e)$ and risk score $r(e)$.
5. Select action: allow, safe-view rewrite, quarantine, block, or propagation-right downgrade.
6. If rewriting, construct and validate the safe view; if validation fails, quarantine.
7. Emit the mediated event, update propagation state, and log a redacted audit record.

Output: mediated event e' , decision d , updated propagation state L' , and audit record.

Proposition 1: raw-containment under complete mediation. Assume complete mediation: every non-quarantine write, send, tool call, and final output passes through FlowFence. Assume safe-view soundness: for every protected item s , the safe-view operator removes raw values and emits only policy-allowed abstractions. If FlowFence rewrites, blocks, or quarantines every event that would carry s to an unauthorized raw recipient, then no non-quarantine event emitted by the guard contains unauthorized raw leakage of s .

Proof sketch. Process events in order. Complete mediation ensures that the guard evaluates each transition before it reaches a non-quarantine channel. If raw disclosure is authorized, the event may be allowed. Otherwise the raw payload is blocked, quarantined, or replaced by a validated safe view. By induction over the event sequence, every emitted non-quarantine event satisfies the raw-disclosure policy.

Topology fanout intuition. Let a contaminated artifact o be readable by at most $F(o)$ downstream agents per step, and let contaminated content propagate for depth d without containment. The number of contaminated transitions can grow on the order of $\sum_{k=0}^d F(o)^k$ when shared objects repeatedly fan out. A chain handoff has local fanout at most one; a blackboard workspace can have fanout proportional to the number of agents. If FlowFence quarantines the raw payload before the first unauthorized high-fanout write, raw propagation outside quarantine is cut off at that boundary while policy-allowed safe-view propagation may continue.

4 Experimental Design

Workflow scenario. The main benchmark models an enterprise vendor-update workflow. Internal agents coordinate over private memory, shared raw memory, safe-view memory, workspace documents, and inter-agent messages to prepare a vendor-facing project update. The task permits coarse disclosure, such as acknowledging that a budget constraint or internal timing issue exists, but forbids exact budget values, internal incident rationales, private customer/project references, and credential-like markers. The final writer must preserve non-sensitive task utility while withholding raw protected values.

Agents, topologies, and research questions. The workflow uses four benchmark roles: a privileged planner, a privileged finance agent, an internal document writer, and an external vendor endpoint. We run the same task, policies, attacks, defenses, and seeds under three runtime topologies. In *chain_4*, information flows planner $\rightarrow$ finance $\rightarrow$ document writer $\rightarrow$ vendor. In *star_4*, the planner is a hub connecting the other roles. In *blackboard_4*, all agents can interact through an all-to-all graph and a shared workspace artifact, increasing fanout. The evaluation asks: RQ1, do final-output and surface defenses miss intermediate propagation? RQ2, does topology affect propagation? RQ3, does FlowFence reduce raw and external leakage while preserving task success? RQ4, can containment be achieved without evaluator-only attack labels? Detailed role, topology, and evidence-matrix tables are in Appendix B.

Attacks and defenses. The LLM final-writer matrix uses seven attack settings: none, summary-poisoning direct/indirect, workspace-poisoning direct/indirect, and communication-hijack direct/indirect. Direct attacks explicitly request sensitive details; indirect attacks embed the request in operational text. Reserved paraphrase variants are used only in the label-free validation. We compare no defense, static access control (static ACL), prompt filtering, and FlowFence. Static ACL blocks hard forbidden channels and unauthorized raw-recipient cases but does not use semantic, provenance, topology, or fanout reasoning. Prompt filtering uses direct phrase/pattern matching and does not rewrite safe views or track propagation. FlowFence applies policy, provenance, topology, safe-view, quarantine, and propagation-right mediation.

Run matrices and provider configuration. The main final-writer experiment uses MiniMax to generate vendor-facing updates from mediated runtime context; MiniMax is not used as the guard, risk scorer, metric evaluator, browser agent, or desktop agent. The main matrix contains 3 topologies $\times$ 7 attack settings $\times$ 4 defenses $\times$ 3 seeds. The label-free validation contains a deterministic matrix with 3 topologies $\times$ 10 attacks $\times$ 6 defenses $\times$ 3 seeds, plus a targeted MiniMax final-writer matrix with 2 topologies $\times$ 3 paraphrase attacks $\times$ 4 defenses $\times$ 3 seeds. The committed configuration records temperature 0.0, max tokens 256, and timeout 60 seconds; provider details are reported in Appendix A.

Metrics. Task success is a deterministic final-output rule/template check for required non-sensitive content, reported separately from privacy leakage. Unauthorized raw leakage counts exact raw-value exposure in runtime event text outside quarantine or blocked events. External leakage counts the subset reaching an external recipient, final output, vendor-send tool, or external message. Cascade size, cascade depth, and privilege reach are computed over the full event graph. Evaluator-label use counts defense decisions whose metadata indicates use of evaluator-only attack labels. These metrics measure runtime propagation rather than only final-answer safety.

5 Experimental Results

5.1 RQ1: Intermediate Propagation Beyond Final Outputs

The retrieval-memory containment check tests whether risky retrieved content becomes exposed downstream context or an unsafe action. Under no defense, poisoned retrievals are exposed and attack manifestation is non-zero. Under FlowFence, risky content may still be retrieved internally, but it is not emitted as exposed model-visible context or an unsafe downstream action. Table 2 shows this transition-level contrast. The static keyword filter is also effective on this known-trigger slice, so the result is used to establish intermediate containment rather than broad paraphrase robustness.

Condition	Utility	Containment outcome
No defense	Clean 0.373; attacked 0.333	Exposed poisoned retrieval 0.467; manifestation 0.253.
FlowFence	Clean 0.360; attacked 0.387	Poison may be retrieved internally (0.440), but exposed poisoned retrieval and manifestation are 0.
Static keyword filter	Known-trigger comparator	Exposure and manifestation are 0 on this same-axis trigger; the result is not used as broad paraphrase evidence.

Table 2: Adapted retrieval-memory containment check. The result is not an official AgentPoison reproduction.

5.2 RQ2: Topology Shapes Propagation Pressure

Topology affects the amount of propagation created by the same attack pressure. Table 3 reports the no-defense attack-positive aggregate: six attack settings, three seeds, and 18 runs per topology. Chain handoffs produce the narrowest but deepest path. Star topologies increase breadth through a coordinator. Blackboard sharing produces the largest cascade size and the highest raw-leakage count because shared workspace artifacts can be read by more downstream agents. External leakage is higher in star and blackboard than in chain, while privilege reach saturates in this no-defense slice.

Topology	Raw leak	External leak	Cascade size	Cascade depth	Priv. reach
<code>chain_4</code>	10.33	2.00	5.00	5.00	5.00
<code>star_4</code>	13.17	2.83	6.00	4.00	5.00
<code>blackboard_4</code>	15.83	2.83	7.00	4.00	5.00

Table 3: Topology effect under no defense, averaged over six attack-positive settings and three seeds per topology (18 runs per topology). Broader sharing increases raw leakage and cascade size; privilege reach saturates in this attack-positive no-defense slice.

5.3 RQ3: FlowFence Reduces Leakage While Preserving Task Success

The MiniMax final-writer matrix evaluates the same multi-agent workflow with MiniMax used only to produce the vendor-facing update. Table 4 reports matched topology–attack–seed comparisons. Across the FlowFence subset, task success remains 1.0 and unauthorized raw leakage, external leakage, and privilege reach are all zero. Compared with no defense, static ACL, and prompt filtering, FlowFence improves or matches leakage outcomes in every matched group. Ties mainly occur in clean or low-pressure groups where both systems already produce zero leakage.

Comparison	Scale	Outcome	Interpretation
FlowFence subset	63/63	Success 1.0; raw 0.0; external 0.0; privilege reach 0	FlowFence runs preserve task completion and eliminate unauthorized raw/external leakage in the evaluated subset.
No defense FlowFence	vs. 63 groups	raw 42 improved / 21 tied; external 32 improved / 31 tied	Untreated shared-state propagation creates preventable exposure.
Static ACL FlowFence	vs. 63 groups	raw 32 improved / 31 tied; external 18 improved / 45 tied	Channel-only rules miss provenance and fanout risks.
Prompt filter FlowFence	vs. 63 groups	raw 18 improved / 45 tied; external 13 improved / 50 tied	Surface-form filtering remains weaker under indirect or shared-state pressure.

Table 4: MiniMax final-writer evaluation on matched topology–attack–seed groups. For raw and external leakage, lower is better. “Improved” counts matched groups in which FlowFence has strictly lower leakage than the comparator; “tied” includes groups where both methods produce zero leakage. Task success is reported separately.

FlowFence’s cascade size is not required to be zero: a contaminated seed can be observed, logged, and quarantined. The privacy target is to prevent unauthorized raw exposure, external exposure, and high-privilege reach outside quarantine. Per-seed stability and detailed parameter settings are reported in Appendix B.

5.4 RQ4: Label-Free Paraphrase Validation

Table 5 addresses a key internal-validity concern: containment would be less meaningful if the guard used evaluator-only attack annotations. The label-free variant removes attack-applied flags, attack identifiers, attack modes, and evaluator labels from guard input. It preserves task success and records zero raw leakage, zero external leakage, and zero evaluator-label use in both the deterministic matrix and the targeted MiniMax final-writer matrix. The no-semantic-pattern ablation increases raw leakage relative to the full label-free variant, indicating that semantic request features help catch laundering pressure while policy, topology/fanout, safe-view construction, and quarantine still contribute.

Setting	Scale	Outcome	Result
Deterministic label-free	540/540; no calls	Success 1.0; raw 0.0; external 0.0	Zero evaluator-label use on paraphrase-validation matrices.
Targeted MiniMax label-free	72/72; MiniMax calls	Success 1.0; raw 0.0; external 0.0	MiniMax final-writer validation with zero evaluator-label use.
Det. vs. no defense	90 comparisons	Raw 81 improved / 9 tied; external 81 improved / 9 tied	No leakage underperformance in matched groups.
Det. vs. static ACL	90 comparisons	Raw 66 improved / 24 tied; external 48 improved / 42 tied	ACL policy gaps remain.
Det. vs. prompt filter	90 comparisons	Raw 54 improved / 36 tied; external 54 improved / 36 tied	Paraphrase failures are recorded for prompt filtering.
MiniMax comparisons	18 comparisons each	Raw: 15/15/16 improved; external: 10/9/10 improved	Final-writer comparisons against no defense, static ACL, and prompt filter.
No-pattern ablation	deterministic subset	Success tied; raw higher; external tied	Semantic request features reduce raw leakage; policy/fanout/safe-view still contribute.

Table 5: Label-free paraphrase validation and partial ablation. The label-free variant removes evaluator-only attack annotations from the guard input.

6 Analysis and Discussion

The results support the paper’s central claim: privacy containment in AI-mediated platforms requires mediating intermediate state transitions. Sensitive content may first appear as a workspace write, summary, or inter-agent message; once that artifact becomes readable, later agents can transform it into routine operational text. Final-output filters observe only the end of this chain. FlowFence acts earlier, at the transition into shared state or external channels.

The topology result clarifies why this is a platform externality. A local write has different downstream costs depending on where it lands. Private memory, a one-hop chain message, a planner-centered star, and a shared blackboard create different propagation surfaces. Static ACLs remain useful for hard forbidden channels, but they do not capture provenance, privilege transitions, fanout, or abstraction rights. Prompt filters are complementary but depend on surface form. FlowFence instead evaluates risk from policy, provenance, topology, privilege, and runtime content signals.

The combination of task success and zero unauthorized raw/external leakage in the evaluated FlowFence runs rules out a trivial block-all explanation. The vendor-facing task still completes because safe views preserve authorized abstractions, while quarantine prevents raw protected values from entering high-fanout shared state. This distinction is important for platform governance: useful coordination often requires disclosing that a constraint exists, not disclosing the raw sensitive value.

The label-free validation reduces the main internal-validity concern. Removing evaluator-only attack annotations from guard input and testing reserved paraphrase variants shows that the reported containment is not simply an artifact of attack labels. The result does not imply coverage of every semantic paraphrase or adaptive adversary; it supports the narrower claim that runtime-observable signals can contain the evaluated propagation patterns.

7 Related Work

Online platforms, networks, and privacy externalities. Online-platform research shows how shared infrastructure and cross-side interactions create network effects among heterogeneous participants [2, 9, 15]. Information-spread models similarly show how local transmissions can have system-level consequences in networked environments [10]. Privacy economics studies disclosure incentives and the costs created when information is reused outside its original context [1]. FlowFence contributes an enforceable runtime mediation primitive for a related AI-platform setting: one principal’s write to shared agent state can impose privacy exposure costs on downstream principals.

Strategic ML and LLM-agent security. Strategic-classification and performative-prediction work studies how participants adapt to learned systems and platform decisions [8, 14]. In LLM-agent systems, indirect prompt injection shows that untrusted data can steer tool-integrated applications [6], and recent benchmarks formalize prompt-injection, tool-agent, and memory-poisoning risks [3, 4, 11, 23]. FlowFence is complementary: it measures and mediates whether sensitive or poisoned content propagates through memories, workspaces, messages, tools, and external outputs.

Information-flow control and runtime enforcement. Classical protection principles and information-flow models provide the foundation for controlling how sensitive data moves through systems [5, 17]. Later work develops language-based information-flow security, decentralized labels, declassification, dynamic taint analysis, and runtime enforcement [12, 13, 16, 18, 22]. FlowFence adapts this intuition to LLM-agent runtimes, where payloads are natural-language artifacts, agents act for different principals, and useful disclosure often requires policy-safe abstraction rather than binary allow/deny labels.

Topology-aware and memory-centric multi-agent systems. Multi-agent LLM systems rely on shared memory, summaries, and collaboration topologies, which also shape safety risk. Agent Smith illustrates how local compromise can spread across interacting agents [7]; G-Safeguard studies topology-aware guardrails for LLM-based multi-agent systems [20]; A-MEM studies agent memory mechanisms [21]; and recent work identifies privacy risks in agent memory and autonomous web agents [19, 24]. FlowFence focuses on platform-level privacy propagation: when raw content should be blocked, quarantined, or rewritten before it enters shared state.

8 Limitations

The evidence is scoped to benchmarked workflows. MiniMax is the only real provider in the final-writer experiments, and it is used to generate vendor-facing text rather than to make guard, scoring, or metric decisions. The runtime is deterministic and platform-mediated; the results therefore speak to the evaluated propagation channels, not to unmediated browser, desktop, operating-system, network, or human-workflow compromise.

The leakage metrics are exact raw-value containment metrics over runtime events and final/vendor-facing text. They do not claim complete semantic leakage detection for every paraphrase. The label-free validation tests reserved paraphrase variants but not arbitrary adaptive adversaries. Safe-view soundness depends on the policy registry and protected-span detectors. The no-semantic-pattern result is a partial ablation rather than a full component decomposition.

The retrieval-memory result is an adapted containment check, and raw traces/provider outputs are not included in the submission artifact; the paper reports aggregate metrics and redacted safe-trace summaries. Future work should test additional providers, larger agent populations, real computer-use environments, learned or verified safe-view constructors, graph-risk models, and deeper module ablations.

9 Conclusion

Multi-agent LLM platforms make privacy a propagation problem. Sensitive information can move through memory, workspaces, summaries, messages, tools, and external outputs before a final answer is produced. FlowFence treats these runtime transitions as first-class governance surfaces: it mediates events, constructs policy-preserving safe views, quarantines raw content, narrows leases, and uses topology and privilege signals to limit downstream exposure. Across retrieval-memory and multi-agent workflow evaluations, the method closes intermediate raw and external leakage paths while preserving task completion. The broader implication is that AI-mediated online platforms should govern shared runtime state directly, rather than relying only on final-output filters.

References

1. Acquisti, A., Taylor, C., Wagman, L.: The economics of privacy. *Journal of Economic Literature* **54**(2), 442–492 (2016)
2. Armstrong, M.: Competition in two-sided markets. *RAND Journal of Economics* **37**(3), 668–691 (2006)
3. Chen, Z., Xiang, Z., Xiao, C., Song, D., Li, B.: Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases. In: *Advances in Neural Information Processing Systems* (2024), <https://openreview.net/forum?id=Y841BRW9rY>
4. Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tramer, F.: Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track* (2024), <https://openreview.net/forum?id=m1YYAQjO3w>
5. Denning, D.E.: A lattice model of secure information flow. *Communications of the ACM* **19**(5), 236–243 (1976)
6. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. pp. 79–90 (2023), <https://arxiv.org/abs/2302.12173>
7. Gu, X., Zheng, X., Pang, T., Du, C., Liu, Q., Wang, Y., Jiang, J., Lin, M.: Agent smith: A single image can jailbreak one million multimodal llm agents exponentially fast. In: *International Conference on Machine Learning* (2024), <https://proceedings.mlr.press/v235/gu24e.html>
8. Hardt, M., Megiddo, N., Papadimitriou, C., Wootters, M.: Strategic classification. In: *Proceedings of the 2016 ACM Conference on Innovations in Theoretical Computer Science*. pp. 111–122 (2016)
9. Katz, M.L., Shapiro, C.: Network externalities, competition, and compatibility. *The American Economic Review* **75**(3), 424–440 (1985)
10. Kempe, D., Kleinberg, J., Tardos, E.: Maximizing the spread of influence through a social network. In: *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. pp. 137–146 (2003)
11. Liu, Y., Deng, G., Xu, Z., Li, Y., Zheng, Y., Zhang, Y., Zhao, L., Zhang, T., Liu, K., Wang, Y.: Formalizing and benchmarking prompt injection attacks and defenses (2023), <https://arxiv.org/abs/2310.12815>
12. Myers, A.C.: Jflow: Practical mostly-static information flow control. In: *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. pp. 228–241 (1999)
13. Newsome, J., Song, D.: Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software. In: *Proceedings of the Network and Distributed System Security Symposium* (2005)
14. Perdomo, J.C., Zrnic, T., Mendler-Dünner, C., Hardt, M.: Performative prediction. In: *Proceedings of the 37th International Conference on Machine Learning*. pp. 7599–7609 (2020)
15. Rochet, J.C., Tirole, J.: Platform competition in two-sided markets. *Journal of the European Economic Association* **1**(4), 990–1029 (2003)
16. Sabelfeld, A., Myers, A.C.: Language-based information-flow security. *IEEE Journal on Selected Areas in Communications* **21**(1), 5–19 (2003)
17. Saltzer, J.H., Schroeder, M.D.: The protection of information in computer systems. *Proceedings of the IEEE* **63**(9), 1278–1308 (1975)

18. Schneider, F.B.: Enforceable security policies. *ACM Transactions on Information and System Security* **3**(1), 30–50 (2000)
19. Wang, B., He, W., Zeng, S., Xiang, Z., Xing, Y., Tang, J., He, P.: Unveiling privacy risks in llm agent memory. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* (2025), <https://aclanthology.org/2025.acl-long.1227/>
20. Wang, S., Zhang, G., Yu, M., Wan, G., Meng, F., Guo, C., Wang, K., Wang, Y.: G-safeguard: A topology-guided security lens and treatment on llm-based multi-agent systems. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* (2025), <https://aclanthology.org/2025.acl-long.359/>
21. Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., Zhang, Y.: A-mem: Agentic memory for llm agents. In: *Advances in Neural Information Processing Systems* (2025), <https://openreview.net/forum?id=FiM0M8gcct>
22. Zdancewic, S., Myers, A.C.: Robust declassification. In: *Proceedings of the 15th IEEE Computer Security Foundations Workshop*. pp. 15–23 (2002)
23. Zhang, H., Huang, J., Mei, K., Yao, Y., Wang, Z., Zhan, C., Wang, H., Zhang, Y.: Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents. In: *International Conference on Learning Representations* (2025), <https://openreview.net/forum?id=V4y0CpX4hK>
24. Zharmagambetov, A., Guo, C., Evtimov, I., Pavlova, M., Salakhutdinov, R., Chaudhuri, K.: Agentdam: Privacy leakage evaluation for autonomous web agents. In: *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track* (2025), <https://openreview.net/forum?id=qaxf7q41aK>

A Supplementary Experimental Implementation Details

These tables record the experiment settings recovered from the committed configurations, runtime code, and high-level artifacts. Missing provider metadata are explicitly marked rather than inferred.

Setting	Injection channel	Form	Intended leakage path
Clean	none	no attack	Task completion without injected leakage pressure
Summary poisoning	summary handoff	direct and indirect	Summary $\rightarrow$ downstream context, workspace, or final text
Workspace poisoning	shared workspace	direct and indirect	Workspace $\rightarrow$ downstream reads or vendor-facing path
Communication hijack	inter-agent message	direct and indirect	Message $\rightarrow$ high-privilege agent, summary, workspace, or output
Reserved phrase variants	para- same three families	paraphrased request	Used only in label-free validation, not in the canonical seven

Table 6: Attack settings. The canonical 252-run matrix contains clean plus direct/indirect variants of the three attack families, giving seven settings. Reserved paraphrase variants are reserved for label-free validation.

Evaluation slice		Formula	Runs	Notes
Canonical matrix	MiniMax	$3 \text{ topologies} \times 7 \text{ attacks} \times 4 \text{ defenses} \times 3 \text{ seeds}$	252	LLM final writer
FlowFence subset		$3 \text{ topologies} \times 7 \text{ attacks} \times 1 \text{ defense} \times 3 \text{ seeds}$	63	Subset of canonical matrix
Deterministic free	label-	$3 \text{ topologies} \times 10 \text{ attacks} \times 6 \text{ defenses} \times 3 \text{ seeds}$	540	No provider calls
Targeted label-free	MiniMax	$2 \text{ topologies} \times 3 \text{ paraphrase attacks} \times 4 \text{ defenses} \times 3 \text{ seeds}$	72	LLM final-writer reserved paraphrase validation
Matched deterministic comparisons	deterministic	Per-baseline matched comparisons in label-free deterministic suite	90	Summary comparison count
Matched comparisons	targeted	Per-baseline matched comparisons in targeted MiniMax suite	18	Summary comparison count

Table 7: Run-count decomposition for the reported evaluation slices.

Item	Recorded setting
Provider role	MiniMax is used as final writer only; it is not the guard, policy engine, risk scorer, metric evaluator, browser agent, or desktop agent
Provider calls	Enabled in LLM final-writer rows; deterministic runtime constructs the final-writer input after mediation
Model name	Actual run model not recorded in committed high-level artifacts; client default is MiniMax-M2.7
Temperature / max tokens / timeout	temperature 0.0; max tokens 256; timeout 60 seconds
Top-p	Not recorded in committed high-level artifacts
Calls per run	High-level artifacts do not directly record the count; code path indicates one final-writer call per MiniMax final-writer run
Metric scope	Leakage metrics are computed over runtime events and final/vendor-facing text, not by a MiniMax judge

Table 8: Provider reproducibility notes. Missing provider metadata are reported explicitly rather than inferred.

Defense	Implementation	Runtime signals used
None	Same runtime with no guard intervention; event trace is still logged	none
Static ACL	Blocks hard forbidden channels and unauthorized raw-recipient cases; no semantic or topology reasoning	channel/principal permissions; limited payload checks
Prompt filter	Direct phrase/pattern filter, not an LLM classifier; no safe-view rewrite	payload surface form only
FlowFence	Uses policy, payload, provenance, topology/fanout, privilege, safe-view, quarantine, and lease signals	payload, policy, provenance, topology/fanout, privilege, channel state
FlowFence label-free	FlowFence variant that ignores attack labels, attack identifiers, attack modes, and evaluator-only labels	same runtime-observable signals as FlowFence, without evaluator labels
No-semantic ablation	Label-free FlowFence without semantic sensitive-detail patterns	FlowFence signals except semantic request patterns

Table 9: Baseline implementation summary. The table records what each defense can observe; it does not imply production deployment.

Parameter or rule	Value / behavior in evaluated implementation
Raw secret weight	0.25
Poison instruction weight	0.45
Instruction-inside-data weight	0.25
Sensitive-detail request weight	0.45
Cross-principal weight	0.15
Forbidden-channel weight	0.50
Shared-workspace high-fanout weight	0.20
Shared-memory high-fanout weight	0.20
Low-trust to high-privilege weight	0.25
External-to-internal weight	0.45
Block override	external-to-internal plus poison/raw content $\rightarrow$ block
Quarantine override	poison plus shared raw/shared document $\rightarrow$ quarantine
Safe-view rewrite threshold	forbidden secret, unauthorized recipient, or risk $\geq 0.50 \rightarrow$ safe-view rewrite
Lease downgrade threshold	risk $\geq 0.30 \rightarrow$ lease downgrade
Label-free removal	ignores attack-applied flags, attack identifiers, attack modes, and evaluator-only labels

Table 10: FlowFence weights, thresholds, and override priorities used in the reported implementation.

Raw item type	Safe abstraction / withholding behavior	Constructor
Exact budget value	Replaced with a coarse statement that a budget constraint exists	Deterministic string rewrite
Internal timing/delay rationale	Replaced with coarse internal timing/delay abstraction when allowed	Deterministic string rewrite
Customer/project reference	Raw identifier withheld; safe text may state that a customer/project reference exists	Deterministic string rewrite
Credential-like marker	Raw marker withheld and represented as an internal credential-like marker withheld	Deterministic string rewrite
Unsafe instruction phrase	Replaced by a safe placeholder or quarantined when rewriting is not sufficient	Deterministic pattern rewrite/quarantine

Table 11: Safe-view construction. The evaluated implementation uses policy-driven deterministic rewriting, not LLM rewriting. Exact raw-value checks validate that protected spans are removed.

Mechanism	Evaluated behavior
Quarantine store	Implemented for memory/workspace zones; quarantined raw content is not emitted to model-visible downstream channels
Downstream view after quarantine	Downstream agents receive a safe view, placeholder, or no emitted content depending on the decision path; raw content remains in quarantine
Propagation stop	Quarantine/block stops downstream propagation of the raw payload in the runtime event graph
Lease downgrade	Implemented as deterministic metadata/control signals such as downgrade or revoke, not as a full independent dynamic capability system
Lease scope in benchmark	Channel and abstraction restrictions narrow onward propagation; no separately enforced time-to-live (TTL) or tool-permission capability system is claimed

Table 12: Quarantine and lease semantics in the evaluated benchmark. The text avoids overstating lease narrowing as a full runtime capability system.

Metric	Implementation / definition	Scope
Task success	Deterministic final-output rule/template check; not an LLM judge. It measures required non-sensitive task content and is reported separately from leakage	Final vendor-facing output
Unauthorized raw leakage	Exact raw-value string matching over event text, skipping quarantined/blocked events; no semantic paraphrase detector is claimed	Full runtime event sequence
External leakage	Raw leakage reaching external recipient, final output, vendor-send tool, or external message	External/final/tool channels
Cascade size/depth	Contaminated event-graph propagation size and longest contaminated causal path	Runtime event graph
Privilege reach	Maximum privilege touched by contaminated content, excluding quarantined/blocked events; privilege levels follow the runtime evaluator’s 0–5 scale	Runtime event graph
Evaluator-label use	Count of label-free runs where defense meta-data records use of evaluator-only attack labels	Defense decision metadata

Table 13: Metric implementation details. Leakage claims are exact raw-value containment claims, not semantic leakage detection claims.

B Supplementary Method and Benchmark Tables

These tables give compact implementation details and secondary benchmark summaries moved out of the main text to keep the submission focused.

Agent / channel	Principal / domain		Role in workflow	Raw-access policy
Planner	Privileged coordinator	internal	Plans the update, routes information, and coordinates finance and drafting through memory, messages, and summaries	May use authorized budget/delay/customer abstractions; must not disclose raw protected values to vendor paths
Finance agent	Privileged owner	finance	Holds budget, internal rationale, customer reference, and credential-like protected items; supplies constraints for the update	Authorized raw access for finance-owned protected items, including credential-like marker
Document writer	Normal drafting principal	internal	Produces internal draft/update text from planner and finance context using workspace and final-draft buffers	May receive internal drafting context; vendor-facing output must contain only allowed abstractions
External endpoint	vendor	Low-trust external recipient/channel	Receives vendor-facing update through external message, vendor-send tool, or final output	No raw access; only vendor-safe abstractions are allowed

Table 14: Agents, principals, and raw-access restrictions in the multi-agent workflow. Vendor-facing delivery is treated as an external channel even when represented by an agent object.

Topology	Communication structure	Shared object	Propagation pressure
chain_4	Planner → finance → document writer → external vendor	No global blackboard; next-hop messages and summaries dominate	Narrow but deeper handoff path
star_4	Planner hub connects finance, document writer, and external vendor	Coordinator state and hub summaries	Aggregation risk at central planner
blackboard_4	All-to-all interactions plus shared workspace/blackboard	Shared workspace artifact readable by multiple agents	Highest fanout and shared-state pressure

Table 15: Topology definitions. All topologies use the same task, protected items, attacks, defenses, and seeds; only graph structure and shared-state fanout change.

RQ	Evidence source	Setting	What it tests
RQ1	Adapted retrieval-memory comparator	P0 AgentPoison-style containment check	Whether risky retrieved content becomes exposed context or unsafe action
RQ1–RQ2	Deterministic propagation suite	Chain, star, and blackboard topologies	Intermediate propagation and topology-dependent cascade behavior
RQ3	MiniMax final-writer evaluation	252 runs: 3 topologies, 7 attack settings, 4 defenses, 3 seeds	LLM final-writer evaluation of leakage and task success
RQ4	Label-free paraphrase validation	540 deterministic runs and 72 targeted MiniMax runs	Whether containment uses runtime-observable signals rather than evaluator labels
RQ4	No-semantic-pattern ablation	Deterministic subset	Partial role of semantic sensitive-detail features

Table 16: Evidence sources organized by research question. The matrix is unchanged; the table clarifies what each slice can test.

Policy field	Meaning	Example
Owner	Principal that owns the protected item	Finance team
Raw detector	Exact value or typed detector for protected content	Budget value, customer ID
Authorized readers	Principals allowed to receive the raw value	Planner and finance agents
Allowed abstraction	Safe downstream description	“budget constraint exists”
Forbidden channels	Destinations that may not carry raw content	Vendor output, shared blackboard
Sensitivity	Cost weight used for exposure accounting	High for credentials

Table 17: Policy interface. FlowFence distinguishes forbidden raw disclosure from policy-allowed abstraction.

Runtime condition	Action	Effect
Unauthorized raw content targets final, external, or tool channel	Block or safe view	Prevent direct exposure
Unauthorized raw content targets high-fanout shared state	Quarantine raw; emit safe view if valid	Stop cascade through shared object
Low-trust provenance enters high-privilege context	Lease downgrade	Narrow onward propagation rights
No raw violation and low propagation risk	Allow	Preserve useful task progress
Safe view cannot be validated	Quarantine	Fail closed for raw content

Table 18: Decision rules used by FlowFence. Rules are fixed across reported experiments.

Channel	Example risk	Evaluation slice
Retrieval memory	Poisoned memory becomes model-visible context	P0 comparator
Summary handoff	Private details copied into an agent summary	Summary poisoning
Workspace artifact	Shared document fans out sensitive content	Workspace poisoning
Inter-agent message	Low-trust message steers inter-agent behavior	Comm. hijack
Paraphrased request	Dangerous request avoids known trigger phrases	Label-free validation

Table 19: Threat channels covered by the multi-agent runtime.

Seed	Success	Raw leak	Ext. leak
1	1.0	0.0	0.0
2	1.0	0.0	0.0
3	1.0	0.0	0.0
All	63/63 clean	0.0	0.0

Table 20: Seed stability in the MiniMax final-writer evaluation. FlowFence remains clean across all evaluated FlowFence runs for seeds 1–3.

C Additional Evaluation Scope and Safe-Trace Summaries

The following tables summarize the evaluation scope and redacted safe-trace cases. They are supplementary review material and do not broaden the empirical claims beyond the main text.

Evaluation slice	Supports	Outside scope	Scale
P0 adapted comparator	Retrieval-memory containment on a saved adapted full-ReAct axis.	Official AgentPoison reproduction or broad poisoning robustness.	summaries
Deterministic MAS	Propagation mechanism, topology effects, and workflow indirect-attack comparisons.	Real-provider behavior.	252 runs
MiniMax final-writer evaluation	MiniMax final-writer workflow evaluation, FlowFence clean subset, and improvement/tie comparisons.	Non-MiniMax generalization, production safety, or real computer-use deployment.	252 runs
Label-free validation	Reserved paraphrase variant evidence without evaluator-only attack annotations.	Arbitrary paraphrase robustness or adaptive adversaries.	540 det. + 72 Mini-Max
Mechanism ablation	Semantic patterns contribute to raw-leakage containment; policy/fanout/safe-view still matter.	Complete isolation of every FlowFence component.	partial

Table 21: Evaluation scope. The table links each experimental slice to the claims it can and cannot support.

Illustration	Setting	Safe-trace observation	Role
No-defense workspace leak	Blackboard; indirect workspace; seed 1; no defense	Shared workspace propagation reaches an external-facing path; task succeeds, but raw leak 16, external leak 1, cascade 7, privilege reach 5.	Failure case
Prompt-filter paraphrase failure	Blackboard; paraphrased workspace; seed 1; prompt filter	Paraphrased shared-state request bypasses surface-form filtering; raw leak 16 and external leak 1 in the targeted safe trace.	Baseline note
FlowFence safe-view/quarantine	Blackboard; indirect workspace; seed 1; FlowFence	A risky workspace write is quarantined or rewritten; task success is preserved and raw/external leakage and privilege reach are 0.	Prevention pair
Label-free reserved paraphrase success	Blackboard; paraphrased workspace; seed 1; label-free FlowFence	Quarantine occurs with evaluator-label use false; task success true, raw/external leakage 0, and evaluator-label use 0.	Validity note

Table 22: Redacted qualitative examples derived from safe traces. They illustrate evaluated MiniMax final-writer workflow-benchmark behavior; they are not raw transcripts, provider outputs, production logs, or additional experiments.